EXPERIMENT 1: BASIC DATABASE CREATION (LIBRARY)

Complete independent practical • MySQL 8.0+ • Database, tables and sample records included

```
-- EXPERIMENT 1: Create a simple Library database
CREATE DATABASE IF NOT EXISTS library;
USE library;

CREATE TABLE book (
    book_id INT PRIMARY KEY,
    book_name VARCHAR(50) NOT NULL,
    author VARCHAR(40),
    price INT
);

INSERT INTO book VALUES
(1, 'Wings of Fire', 'A. P. J. Abdul Kalam', 250),
(2, 'The Alchemist', 'Paulo Coelho', 300),
(3, 'Ikigai', 'Hector Garcia', 200);

SELECT * FROM book;
```

EXPERIMENT 2: ER/EER TO RELATIONAL SCHEMA (LIBRARY)

Complete independent practical • MySQL 8.0+ • Database, tables and sample records included

```
-- EXPERIMENT 2: Map a Library ER model into relations
CREATE DATABASE IF NOT EXISTS library;
USE library;

CREATE TABLE member (
    member_id INT PRIMARY KEY,
    member_name VARCHAR(40) NOT NULL,
    phone VARCHAR(15)
);

CREATE TABLE book (
    book_id INT PRIMARY KEY,
    book_name VARCHAR(50) NOT NULL,
    author VARCHAR(40)
);

-- One member can borrow many books over time.
-- BORROW is a relationship table connecting MEMBER and BOOK.
CREATE TABLE borrow (
    borrow_id INT PRIMARY KEY,
    member_id INT,
    book_id INT,
    borrow_date DATE,
    FOREIGN KEY (member_id) REFERENCES member(member_id),
    FOREIGN KEY (book_id) REFERENCES book(book_id)
);

INSERT INTO member VALUES
(1, 'Asha', '9876543210'),
(2, 'Ravi', '9876501234');

INSERT INTO book VALUES
(101, 'Wings of Fire', 'A. P. J. Abdul Kalam'),
(102, 'The Alchemist', 'Paulo Coelho');

INSERT INTO borrow VALUES
(1, 1, 101, '2026-10-01'),
(2, 2, 102, '2026-10-02');

-- Relational schema:
-- MEMBER(member_id PK, member_name, phone)
-- BOOK(book_id PK, book_name, author)
-- BORROW(borrow_id PK, member_id FK, book_id FK, borrow_date)
SELECT * FROM member;
SELECT * FROM book;
SELECT * FROM borrow;
```

EXPERIMENT 3: DDL AND INTEGRITY CONSTRAINTS (SHOP)

Complete independent practical • MySQL 8.0+ • Database, tables and sample records included

```
-- EXPERIMENT 3: DDL and constraints using a Shop database
CREATE DATABASE IF NOT EXISTS shop;
USE shop;

CREATE TABLE product (
    product_id INT PRIMARY KEY,
    product_name VARCHAR(40) NOT NULL,
    price DECIMAL(8,2) CHECK (price > 0),
    product_code VARCHAR(20) UNIQUE,
    stock INT DEFAULT 0
);

CREATE TABLE customer (
    customer_id INT PRIMARY KEY,
    customer_name VARCHAR(40) NOT NULL,
    email VARCHAR(60) UNIQUE
);

INSERT INTO product VALUES
(1, 'Pen', 10.00, 'PEN01', 100),
(2, 'Notebook', 50.00, 'NOTE01', 60);

INSERT INTO customer VALUES
(1, 'Amit', 'amit@example.com'),
(2, 'Neha', 'neha@example.com');

-- ALTER: add column
ALTER TABLE product ADD category VARCHAR(30);

-- ALTER: modify column
ALTER TABLE product MODIFY product_name VARCHAR(60) NOT NULL;

-- ALTER: add constraint
ALTER TABLE product ADD CONSTRAINT chk_stock CHECK (stock >= 0);

-- ALTER: drop constraint
ALTER TABLE product DROP CHECK chk_stock;

-- ALTER: drop column
ALTER TABLE product DROP COLUMN category;

-- Test RENAME, TRUNCATE and DROP
CREATE TABLE test_item (id INT PRIMARY KEY, item_name VARCHAR(30));
INSERT INTO test_item VALUES (1, 'Sample');
RENAME TABLE test_item TO sample_item;
TRUNCATE TABLE sample_item;
DROP TABLE sample_item;

SELECT * FROM product;
SHOW TABLES;
```

EXPERIMENT 4: DML (BANK ACCOUNT)

Complete independent practical • MySQL 8.0+ • Database, tables and sample records included

```
-- EXPERIMENT 4: INSERT, UPDATE and DELETE in a Bank database
CREATE DATABASE IF NOT EXISTS bank;
USE bank;

CREATE TABLE account (
    account_no INT PRIMARY KEY,
    holder_name VARCHAR(40) NOT NULL,
    balance DECIMAL(10,2) DEFAULT 0
);

INSERT INTO account VALUES
(1001, 'Amit', 5000.00),
(1002, 'Neha', 8000.00),
(1003, 'Ravi', 3000.00);

SELECT * FROM account;

-- INSERT a new account
INSERT INTO account VALUES (1004, 'Kiran', 4500.00);
SELECT * FROM account;

-- UPDATE balance
UPDATE account
SET balance = balance + 1000
WHERE account_no = 1004;
SELECT * FROM account;

-- DELETE an account record
DELETE FROM account
WHERE account_no = 1003;
SELECT * FROM account;
```

EXPERIMENT 5: DCL (USER PERMISSIONS)

Complete independent practical • MySQL 8.0+ • Database, tables and sample records included

```
-- EXPERIMENT 5: GRANT and REVOKE on a School database
CREATE DATABASE IF NOT EXISTS school;
USE school;

CREATE TABLE teacher (
    teacher_id INT PRIMARY KEY,
    teacher_name VARCHAR(40) NOT NULL,
    subject VARCHAR(30)
);

INSERT INTO teacher VALUES
(1, 'Meena', 'Maths'),
(2, 'Raj', 'Science');

-- User creation usually requires MySQL administrator privileges
CREATE USER IF NOT EXISTS 'school_user'@'localhost'
IDENTIFIED BY 'School123';

-- Give permission to view and add teacher records
GRANT SELECT, INSERT
ON school.teacher TO 'school_user'@'localhost';

SHOW GRANTS FOR 'school_user'@'localhost';

-- Remove INSERT permission
REVOKE INSERT
ON school.teacher FROM 'school_user'@'localhost';

SHOW GRANTS FOR 'school_user'@'localhost';

-- Optional: delete this user after the practical
-- DROP USER 'school_user'@'localhost';
```

EXPERIMENT 6: SQL QUERIES AND AGGREGATE FUNCTIONS (EMPLOYEE)

Complete independent practical • MySQL 8.0+ • Database, tables and sample records included

```
-- EXPERIMENT 6: SELECT and aggregate functions in an Office database
CREATE DATABASE IF NOT EXISTS office;
USE office;

CREATE TABLE employee (
    emp_id INT PRIMARY KEY,
    emp_name VARCHAR(40) NOT NULL,
    salary INT,
    department VARCHAR(30)
);

INSERT INTO employee VALUES
(1, 'Amit', 25000, 'Sales'),
(2, 'Neha', 35000, 'IT'),
(3, 'Ravi', 30000, 'Sales'),
(4, 'Kiran', 40000, 'HR'),
(5, 'Sonal', 45000, 'IT');

-- Basic SELECT
SELECT * FROM employee;
SELECT emp_name, salary FROM employee;

-- WHERE, AND, OR, BETWEEN, IN and LIKE
SELECT * FROM employee WHERE salary > 30000;
SELECT * FROM employee WHERE salary > 25000 AND department = 'IT';
SELECT * FROM employee WHERE department = 'IT' OR department = 'HR';
SELECT * FROM employee WHERE salary BETWEEN 25000 AND 40000;
SELECT * FROM employee WHERE department IN ('IT', 'Sales');
SELECT * FROM employee WHERE emp_name LIKE 'S%';

-- DISTINCT and ORDER BY
SELECT DISTINCT department FROM employee;
SELECT * FROM employee ORDER BY emp_name ASC;
SELECT * FROM employee ORDER BY salary DESC;

-- Aggregate functions
SELECT COUNT(*) AS total_employees FROM employee;
SELECT SUM(salary) AS total_salary FROM employee;
SELECT AVG(salary) AS average_salary FROM employee;
SELECT MIN(salary) AS lowest_salary FROM employee;
SELECT MAX(salary) AS highest_salary FROM employee;

-- GROUP BY and HAVING
SELECT department, COUNT(*) AS employee_count
FROM employee GROUP BY department;

SELECT department, AVG(salary) AS average_salary
FROM employee GROUP BY department
HAVING AVG(salary) > 30000;
```

EXPERIMENT 7: JOIN OPERATIONS (CUSTOMER AND ORDERS)

Complete independent practical • MySQL 8.0+ • Database, tables and sample records included

```
-- EXPERIMENT 7: Join types using a simple Shop database
CREATE DATABASE IF NOT EXISTS shop;
USE shop;

CREATE TABLE customer (
    customer_id INT PRIMARY KEY,
    customer_name VARCHAR(40) NOT NULL,
    city VARCHAR(30)
);

CREATE TABLE orders (
    order_id INT PRIMARY KEY,
    customer_id INT,
    amount INT,
    FOREIGN KEY (customer_id) REFERENCES customer(customer_id)
);

INSERT INTO customer VALUES
(1, 'Amit', 'Mumbai'),
(2, 'Neha', 'Pune'),
(3, 'Ravi', 'Nashik'),
(4, 'Kiran', 'Thane');

INSERT INTO orders VALUES
(101, 1, 500),
(102, 2, 900),
(103, 1, 300),
(104, 3, 700);

-- Equijoin / INNER JOIN
SELECT customer.customer_name, orders.order_id, orders.amount
FROM customer INNER JOIN orders
ON customer.customer_id = orders.customer_id;

-- Non-equijoin (amount is compared with a range)
SELECT customer.customer_name, orders.amount
FROM customer JOIN orders
ON orders.amount > 400 AND orders.amount < 800;

-- Self-join: customers living in the same city
SELECT a.customer_name AS customer1, b.customer_name AS customer2,
       a.city
FROM customer a JOIN customer b
ON a.city = b.city AND a.customer_id < b.customer_id;

-- LEFT OUTER JOIN
SELECT customer.customer_name, orders.order_id
FROM customer LEFT JOIN orders
ON customer.customer_id = orders.customer_id;

-- RIGHT OUTER JOIN
SELECT customer.customer_name, orders.order_id
FROM customer RIGHT JOIN orders
ON customer.customer_id = orders.customer_id;

-- CROSS JOIN
SELECT customer.customer_name, orders.order_id
FROM customer CROSS JOIN orders;
```

EXPERIMENT 8: VIEWS AND TRIGGERS (PRODUCT AUDIT)

Complete independent practical • MySQL 8.0+ • Database, tables and sample records included

```
-- EXPERIMENT 8: View and triggers in a Store database
CREATE DATABASE IF NOT EXISTS store;
USE store;

CREATE TABLE product (
    product_id INT PRIMARY KEY,
    product_name VARCHAR(40) NOT NULL,
    price INT
);

INSERT INTO product VALUES
(1, 'Pen', 10),
(2, 'Notebook', 50),
(3, 'Bag', 500);

-- Create a simple view
CREATE VIEW product_view AS
SELECT product_id, product_name, price
FROM product;

SELECT * FROM product_view;

-- Log table to record changes
CREATE TABLE product_log (
    log_id INT AUTO_INCREMENT PRIMARY KEY,
    action_name VARCHAR(20),
    product_id INT,
    action_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

DELIMITER //

CREATE TRIGGER product_after_insert
AFTER INSERT ON product FOR EACH ROW
BEGIN
    INSERT INTO product_log (action_name, product_id)
    VALUES ('INSERT', NEW.product_id);
END//

CREATE TRIGGER product_after_update
AFTER UPDATE ON product FOR EACH ROW
BEGIN
    INSERT INTO product_log (action_name, product_id)
    VALUES ('UPDATE', NEW.product_id);
END//

CREATE TRIGGER product_after_delete
AFTER DELETE ON product FOR EACH ROW
BEGIN
    INSERT INTO product_log (action_name, product_id)
    VALUES ('DELETE', OLD.product_id);
END//

DELIMITER ;

-- Test all three triggers
INSERT INTO product VALUES (4, 'Pencil', 5);
UPDATE product SET price = 6 WHERE product_id = 4;
DELETE FROM product WHERE product_id = 4;

SELECT * FROM product_log;
SHOW TRIGGERS;
```

EXPERIMENT 9: TRANSACTIONS AND CONCURRENCY (WALLET)

Complete independent practical • MySQL 8.0+ • Database, tables and sample records included

```
-- EXPERIMENT 9: Transaction, COMMIT, ROLLBACK and locking
CREATE DATABASE IF NOT EXISTS wallet_db;
USE wallet_db;

CREATE TABLE wallet (
    wallet_id INT PRIMARY KEY,
    owner_name VARCHAR(40) NOT NULL,
    balance INT NOT NULL
) ENGINE = InnoDB;

INSERT INTO wallet VALUES
(1, 'Amit', 5000),
(2, 'Neha', 7000);

SELECT * FROM wallet;

-- Transaction: transfer 500 from Amit to Neha
START TRANSACTION;

UPDATE wallet SET balance = balance - 500 WHERE wallet_id = 1;
UPDATE wallet SET balance = balance + 500 WHERE wallet_id = 2;

SELECT * FROM wallet;

COMMIT;

SELECT * FROM wallet;

-- Demonstrate ROLLBACK: temporary change will be cancelled
START TRANSACTION;

UPDATE wallet SET balance = balance - 1000 WHERE wallet_id = 1;
SELECT * FROM wallet;

ROLLBACK;

SELECT * FROM wallet;

-- Concurrency / row lock demonstration (Session 1)
START TRANSACTION;

SELECT * FROM wallet
WHERE wallet_id = 1
FOR UPDATE;

-- Keep Session 1 open. In Session 2, run:
-- UPDATE wallet SET balance = balance + 100 WHERE wallet_id = 1;
-- Session 2 waits while Session 1 holds the lock.

-- Return to Session 1 and release the lock:
COMMIT;

-- Session 2 can now finish its UPDATE.
-- Verify:
SELECT * FROM wallet;
```